

Reliable AI-Assisted Grading: A Multi-Agent System for Open-Ended Assignments

Aarya Rajgor, Satyaa Sudarshan Guruswaamy Sethuraaman, and Muskan Jain
AI Grading Agent Project | August 2026

Abstract—This report presents an AI-assisted grading system for open-ended assignments. The implementation combines rubric-grounded LLM judgment with mathematical verification, an independent critic, bounded revision, human escalation, and an auditable approval workflow. It is designed to address the parts of grading that simple answer matching cannot handle: recognizing a valid method, assigning partial credit, and producing feedback tied to recorded evidence. The report examines the system design, code-level reliability controls, data and retrieval quality, label-free evaluation, and measured API usage from five real end-to-end assignments.

Keywords—*generative AI, automated grading, multi-agent systems, partial credit, RAG, evaluation*

I. PROBLEM DEFINITION AND GENAI FIT

Open-ended assignments expose the limits of traditional automated grading. A numerical answer checker can determine whether a final value matches a key, but it cannot reliably assess the quality of a student's process. Instructors routinely award credit for a correct setup, a valid intermediate method, or a response that is conceptually sound but contains a small algebraic error. The same issue is more pronounced for proofs and short-answer questions, where the work must be evaluated against a method, evidence, and a rubric rather than a single canonical string.

The project addresses this problem as a decision-support system for instructors. Its goal is not to replace academic judgment; it is to reduce the time spent on routine first-pass evaluation while preserving the cases that deserve human attention. The system reads the problem statement, reference solution, rubric criteria, student work, and final answer as a combined context. It then records why a score was proposed, what objective evidence exists, whether a critic agreed, and whether a human decision is still needed.

Generative AI is appropriate because language models can interpret alternate explanations and incomplete work in ways that rule-based logic cannot. The model can identify a valid technique even if notation differs from the reference, connect a missing step to the relevant rubric criterion, and draft feedback in plain language. That flexibility carries risk: the model can also sound confident when it is wrong. The project therefore treats the LLM as one component in a reliability architecture, not as an autonomous final grader.

II. BASELINE COMPARISON

The natural baseline is a single LLM prompt that receives a rubric and a submission and returns a score with feedback. This baseline is inexpensive and fast, but it has no independent mechanism for challenging a questionable decision. A borderline response may be over-scored because the model recognizes familiar words, or under-scored because the response does not follow the reference solution's exact path. The baseline also makes it difficult to distinguish a tool-confirmed fact from an interpretive judgment.

The implemented pipeline adds four layers around that baseline. First, it uses a SymPy-backed equivalence check for mathematical final answers when such a check is meaningful. Second, it requires the grader to keep problem-level evidence and a partial-credit rationale. Third, it asks an independent critic to review nontrivial judgments. Finally, it permits one bounded revision; a persistent disagreement is escalated to an instructor instead of being resolved by more agent debate. The additional calls increase latency and token usage, but they produce a more defensible result.

The dropped-term expansion case illustrates the difference. A student who writes $x^2 + 9$ for $(x + 3)^2$ has recognized the broad form of a square but missed the $6x$ middle term. A score of zero ignores meaningful work; full credit ignores a substantive mathematical error. The system can use rubric evidence to explain partial credit, permit a critic to challenge the proposed score, and send unresolved cases to review. A single-prompt baseline has no comparable escalation path.

III. APPLICATION AND TECHNICAL DEPTH OF GENAI TECHNIQUES

A. Three-Lane Architecture and Contracts

The codebase separates the workflow into three lanes. Lane P1 handles ingestion, textbook retrieval, solution development, rubric drafting, context construction, and persistence. Lane P2 contains the grading engine, grader, critic, verification tools, batch execution, and grade traces. Lane P3 produces feedback, exposes the human review workflow, stores audit entries, and computes evaluation metrics. Facade modules provide stable public APIs while specialized helper modules implement the details beneath them.

A frozen `contracts.py` layer is central to the architecture. It defines artifacts such as Assignment, Problem, Rubric, GradingContext, ProblemGrade, Grade, Step, and Trace. The GradingContext contract makes P1 responsible for selecting and

token-budgeting the reference material, while P2 is responsible only for framing that curated context for the grader and critic. This separation prevents each stage from silently re-fetching or re-trimming inputs and makes the reasoning path easier to test.

The contracts also enforce data quality at the object level. A partial score requires a partial-credit reason; a score cannot exceed the available points; a no-answer or ungradeable outcome cannot carry credit; and a published or approved grade must record an approver. The trace envelope captures stop reason, critic agreement, revision count, tokens, latency, and open-ended step data. This keeps the evaluation schema stable while allowing new trace details to be added without breaking the review layer.

B. Grader, Critic, and Verification

The core P2 engine follows an ACT–REASON–CRITIQUE–REVISION loop. ACT records the verifier result. REASON records the grader's proposed outcome, evidence, and partial-credit rationale. CRITIQUE records the independent critic's response. If the critic disagrees, REVISION gives the grader one opportunity to respond to the critique, followed by a recheck. A remaining disagreement marks the problem and the grade as escalated. This is deliberately finite: the system acknowledges uncertainty rather than spending an unbounded number of calls attempting to manufacture consensus.

The verifier is purposefully limited. SymPy can establish equivalence for many mathematical expressions, but it cannot establish whether a proof is persuasive or whether a short answer demonstrates a concept. For unsupported problem types, the verifier returns not applicable rather than a misleading true or false result. The grading context can carry a per-problem type, allowing math, proof, and short-answer questions in a mixed assignment to use different verification behavior.

Critic independence is treated as a design problem rather than a cosmetic feature. The configuration can use a different critic model or temperature, and the evaluation layer tracks the rate at which the critic agrees. A critic that agrees in every case may be a useful observation, but it is not proof that the original grades are perfect. The project explicitly flags near-universal agreement as a reason to inspect whether the critic is functioning as a real check rather than rubber-stamping the grader.

C. Human Review, Feedback, and Security

The final lane keeps the instructor inside the loop. An override requires a reason and creates an audit entry containing the prior score, new score, reviewer, and rationale. An escalated grade cannot be finalized until the affected disagreement has been resolved. Guard parameters prevent an override or approval from being written to a grade belonging to a different submission. These write-time checks are important because a polished interface alone cannot protect student records when a selection state is wrong.

The student-facing layer is intentionally narrower than the internal trace. Feedback is generated from the recorded score and evidence, not from unrestricted rubric or model text. Tests ensure that internal escalation messages and offline-model placeholders do not leak into feedback. Follow-up chat is grounded in the student's recorded grade and, when available, the approved assignment context; without registered evidence, it fails closed rather than inventing an explanation. The broader application supports admin, instructor, and student roles, course ownership and enrollment, a student portal, and regrade requests.

D. Failure Handling and Operational Boundaries

The repository distinguishes expected grading outcomes from system failures. No answer, ungradeable answer, and escalation are captured as data in a ProblemGrade or Grade; they are not thrown as exceptions that can terminate a batch. ParseError, ToolError, and ModelError represent genuine failures instead. The batch boundary can isolate one failing submission so that a malformed file, unavailable model call, or unexpected verification failure does not prevent the rest of a class from being processed. This distinction is operationally important: a student who submits no answer needs a clear record and possibly a zero, whereas a broken external service needs a retry, a visible warning, and no invented grade.

The project also preserves a deterministic offline fallback for development and demonstrations when a BYOK model is not configured. The interface makes this state explicit rather than silently returning model-like results. In a production use, the fallback should not be confused with a valid substitute for LLM reasoning; its role is to keep wiring, storage, and review workflows testable without an API key. Similarly, solution-development disagreement is flagged for human review instead of being automatically approved, preserving the same reliability principle upstream of grading.

IV. DATA, KNOWLEDGE SOURCES, AND DATASET QUALITY

The two principal knowledge sources are instructor-provided assessment material and a local textbook corpus. Assignments and submissions supply the direct grading evidence. The textbook directory contains algebra, calculus, and linear-algebra material used to retrieve a potentially relevant method for solution development, rubric creation, and feedback. Retrieval is grounding, not copying: the model is instructed to reason from useful course material and not to paste it into an answer or feedback message.

The retrieval code reflects practical dataset-quality work. It chunks source files into approximately 900-character windows with 120-character overlap and snaps boundaries to whitespace so that a word or procedure is not cut in half. It uses Chroma when available and has a deterministic word-overlap fallback when it is not. The index has a fingerprint cache so a textbook

corpus is not needlessly re-indexed on every request. The application can also persist textbook chunks and rehydrate a missing local corpus from the database.

The important failure mode is retrieval bleed. In a small corpus, a nearest-neighbor system may return an unrelated passage simply because it is the closest available option. A linear-algebra passage should not be placed in a prompt for a basic algebra expansion merely to fill a retrieval slot. The implementation gates candidates on significant-word overlap, labels retrieved sources, and returns no context when the relevance signal is weak. This is a useful design lesson: the absence of retrieval evidence is safer than a plausible-looking but irrelevant source.

The data model also considers record quality and privacy. A submission has an anonymizable student label, and may carry a stable student identifier so two students with the same display name cannot be confused. Database stores are separated by lane but shared through the configured database URL. SQLite supports local development; Docker Compose starts a health-checked PostgreSQL service for deployment. The repository uses BYOK configuration through environment variables, includes an offline fallback for local exploration, and warns visibly when no real model is configured.

V. EVALUATION PIPELINE

The evaluation layer is intentionally label-free. It aggregates answer-match signals, evidence grounding, critic outcomes, total-score variation, latency, and token usage across grading runs. These indicators describe whether the system is producing traceable work and how much the reliability controls cost. They do not prove that an automated score is correct. A meaningful accuracy study would require a held-out set of instructor-scored assignments and a protocol for comparing final scores, partial-credit decisions, feedback usefulness, and escalation choices.

A separate real end-to-end run provides operational evidence. One three-problem algebra assignment was graded for five submissions with a mix of fully correct, partially correct, and wrong work. The run produced an answer-match rate of 86.7%, a grounding rate of 100%, a critic-agreement rate of 100%, a score standard deviation of 2.32, average latency of 5,782.8 ms, and average token usage of 512.8. The full critic-agreement rate should not be read as an unconditional success: the project correctly treats it as a signal to inspect critic independence.

A trace from a submission that triggered critic disagreement demonstrates the bounded loop in concrete terms. It contained three ACT steps for mathematical checks, four REASON passes because one occurred after revision, two CRITIQUE steps for the initial review and recheck, and one REVISION step. This trace format makes the extra work caused by a contested grade visible, which is valuable when instructors decide whether additional automation cost is justified.

A. Measured Token and API Usage

Five further assignments were run end to end through the configured real BYOK model, covering linear equations, polynomials, systems, quadratics, and radicals. Every run executed ingestion, solution development, rubric drafting, and grading of two submissions. The token figures below came from the provider SDK usage object for each call, not from the application's approximate text-length estimator. Together the five runs used 25,973 input tokens and 6,809 output tokens, or 32,782 total, across 67 API calls.

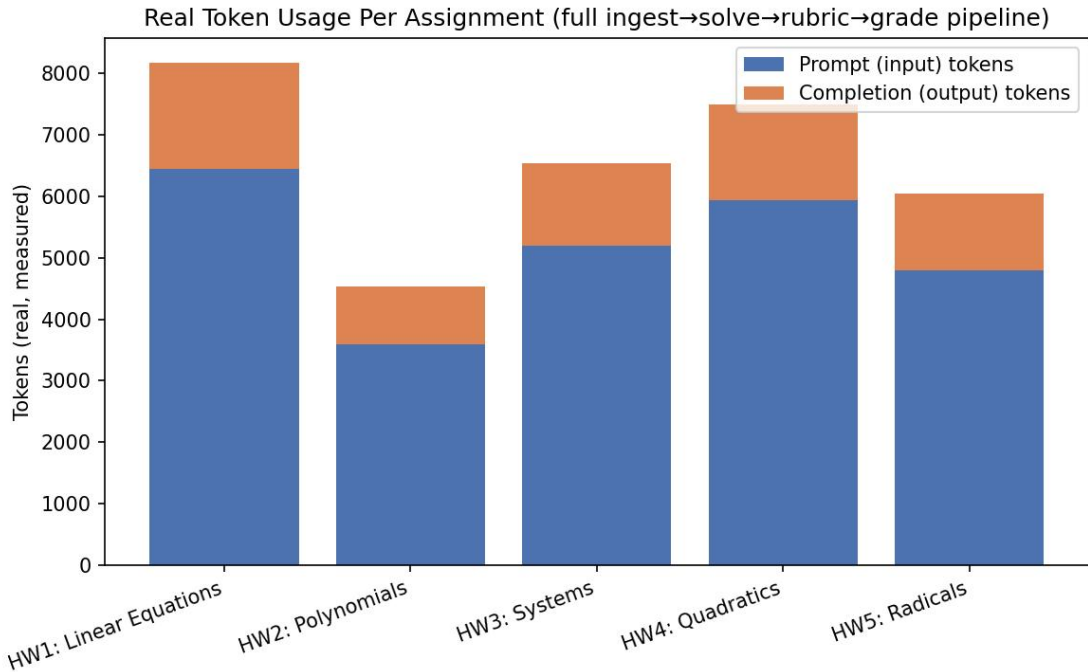


Figure 1. Measured prompt and completion token usage for five complete assignment runs. HW1 required 8,169 total tokens; HW2 required 4,529.

The input portion dominates each stack because the pipeline carries assignment text, reference material, rubrics, and student work into several stages. Completion usage is smaller but varies with the generated solution, rubric, feedback, and any revision. The variation is expected: the assignments differ in prompt length and the number of calls needed for their course context. Token count is therefore a better direct cost indicator than the number of API calls by itself.

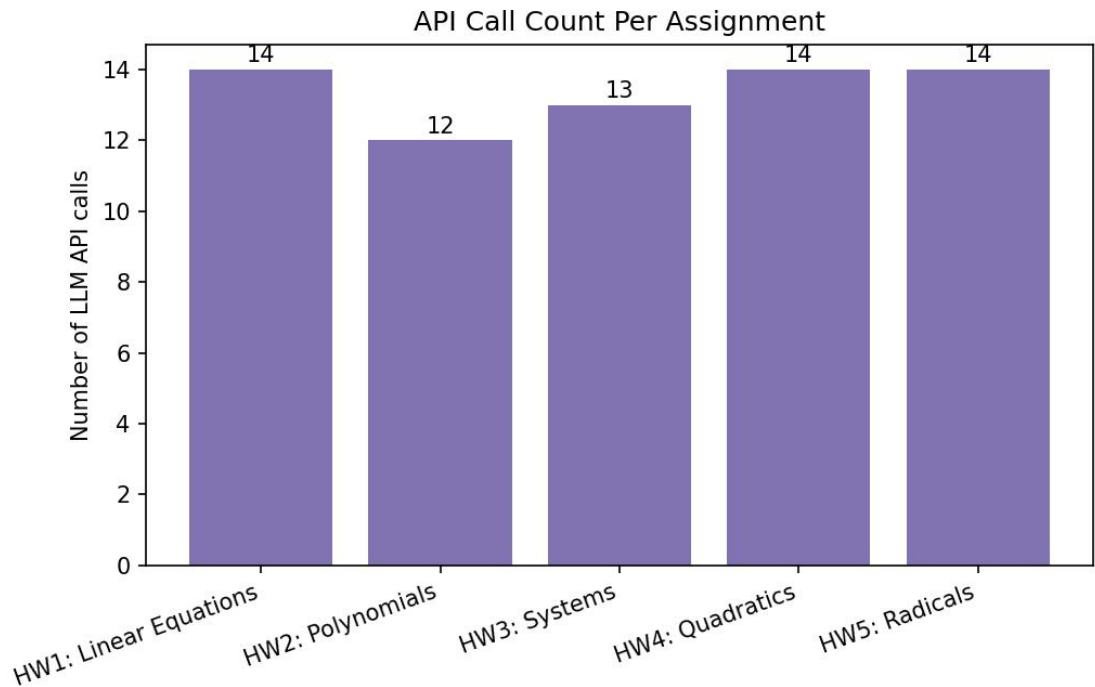


Figure 2. API call count per assignment. The 12-14 calls include classification, solution generation and checking, rubric drafting, grading, critic review, and occasional revision.

B. Cost Interpretation

The cost plots use an illustrative conversion of \$0.15 per million input tokens and \$0.60 per million output tokens. The charted dollar values must not be presented as a verified provider price for the configured model; they are a transparent scenario calculation. The measured token counts are the empirical result. To obtain an authoritative production cost, the same input and output totals should be multiplied by the provider's applicable price at the time of deployment.

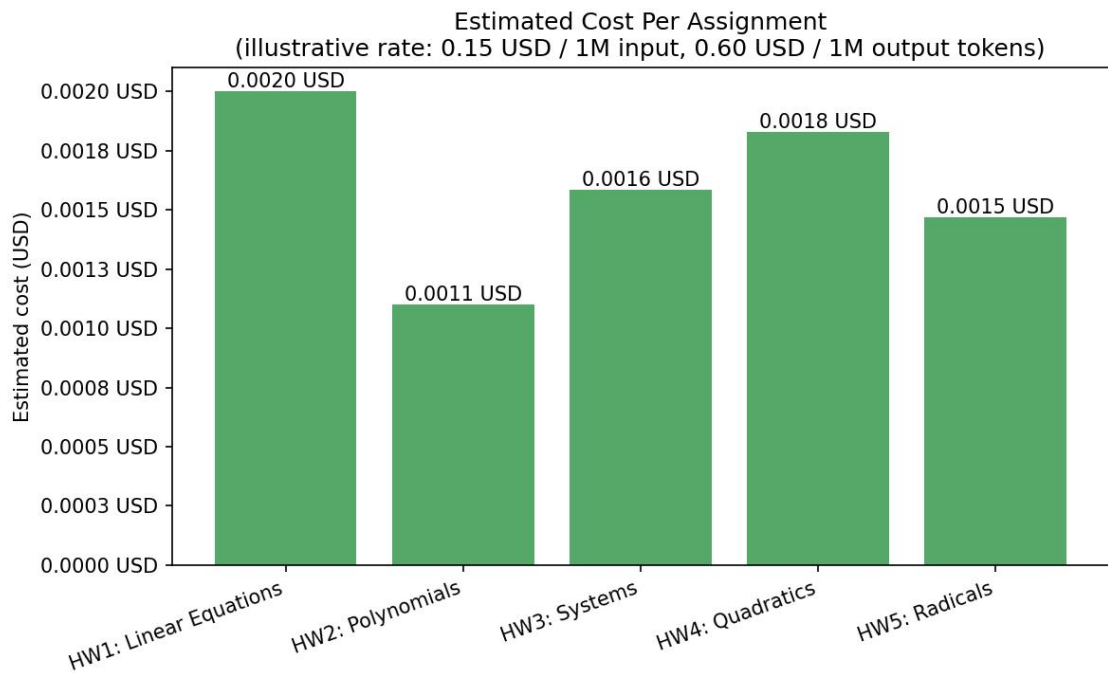


Figure 3. Illustrative estimated cost per assignment using the stated input/output token rates. The measured token totals, not these rates, are the primary evaluation artifact.

At the illustrative rate, the five assignments sum to \$0.0080. The cumulative plot makes the unit economics easy to inspect: each point represents another complete ingest-to-grade run, including the reliability mechanisms rather than just one scoring prompt. This is a useful comparison against a single-call baseline because it makes the added cost of solution development, rubric drafting, and critic review explicit.

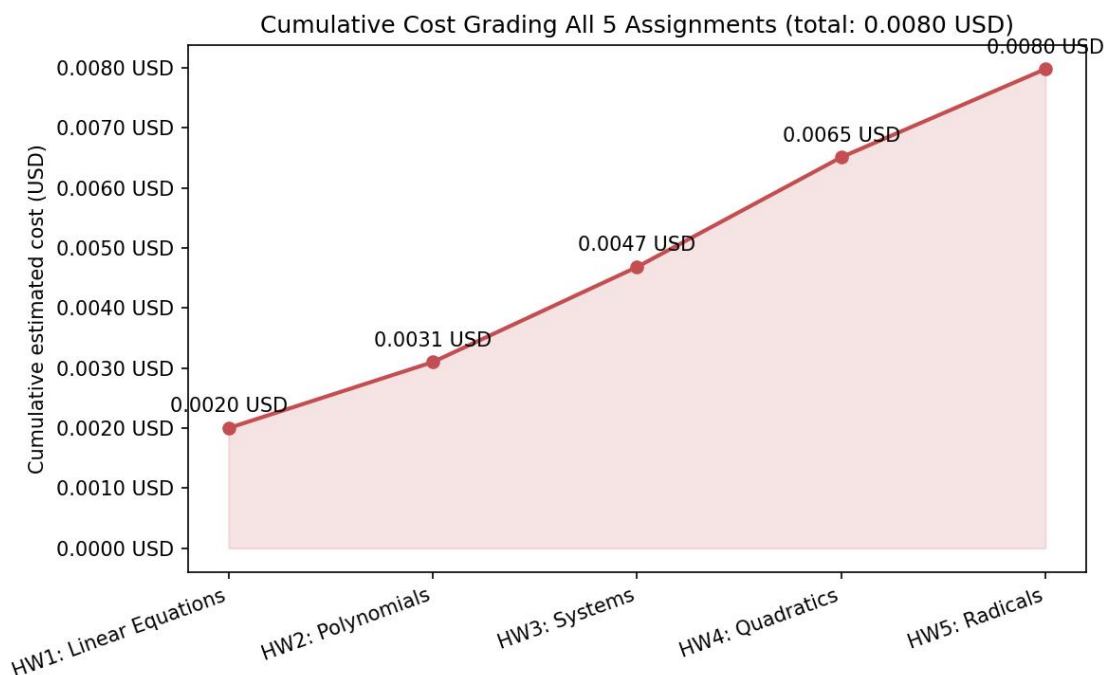


Figure 4. Cumulative illustrative cost of the five end-to-end assignment runs, reaching \$0.0080 under the stated scenario rate.

C. Reading the Run as a System Experiment

The measured run was designed to exercise the full chain rather than only the final grading prompt. For every assignment, the system first handled the assignment and course context, then developed or checked a solution, drafted a rubric, and graded two submissions. As a result, the token and call charts represent the cost of the complete workflow an instructor would actually use, not a misleadingly narrow price for producing a single score. The highest observed token total occurred for linear equations, while the lowest occurred for polynomials; this demonstrates that prompt size, generated solution length, and revision behavior affect cost more directly than a simple subject label.

The five-assignment run is also useful for separating fixed and variable overhead. Classification, solution work, and rubric drafting contribute a base number of calls per assignment. Critic review and revision depend more directly on the submissions. A system operator can use the trace and cost data together: a high call count with modest tokens suggests orchestration latency, while a high token count with similar calls suggests larger contexts or more verbose generation. The present data do not support a universal cost prediction, but they do show that usage can be measured at the point of the provider call and associated with a concrete assignment pipeline.

VI. EXPERIMENTS PERFORMED

The repository's tests go beyond a happy-path demo. They cover ingestion, text sanitization and prompt-injection resistance, solution development, rubric construction, context budgeting, symbolic verification, grader behavior, critic behavior, batch isolation, storage, authentication, role gating, student portal views, and session state. The test organization maps closely to the three-lane design, allowing a regression in one layer to be isolated without assuming that the entire pipeline is correct because a screen renders.

The partial-credit experiments include correct, wrong, and incomplete mathematical answers. In the dropped-middle-term expansion case, the recorded feedback identifies the missing cross term rather than supplying generic praise or a bare score. Mixed-assignment tests ensure that an individual problem's classified type overrides a submission-level default, so a proof question is not incorrectly passed to SymPy simply because another question uses algebra.

Review experiments exercise the human override to approval path. A human score change retains the original AI evidence, places the instructor rationale in the audit log, and clears only the escalation associated with the reviewed problem. If a second

problem in the same submission remains disputed, approval remains blocked. This prevents a broad flag from being cleared accidentally after an unrelated edit. The system also tests the regrade path and the refusal of cross-submission writes.

The feedback experiments test groundedness rather than prose style alone. Student chat is supplied only with the recorded grade evidence and, when registered, the relevant assignment context. It is instructed not to freelance an answer beyond the available evidence. The tests confirm that it can explain a lost point, retain short conversation history for the same grade, reset that history after a regrade, and decline to answer when no suitable context has been registered.

A. Limitations and Threats to Validity

The experiments demonstrate implementation behavior, not a completed educational-effectiveness study. The real grading batch is small, the assignments are algebra-focused, and the label-free metrics do not establish agreement with instructor scores. A 100% grounding rate means that the system stored evidence; it does not show that the evidence was sufficient or pedagogically helpful. An 86.7% answer-match rate is restricted to the cases where an objective answer check applies, and it should not be generalized to proof or short-answer grading. These distinctions prevent operational telemetry from being overstated as proof of model quality.

There are additional human factors to evaluate. Instructors need to know whether a proposed score reduces workload without creating new review burden; students need feedback that is accurate, actionable, and fair. Future work should use a held-out dataset with double-marked instructor labels, compare the baseline and multi-agent approaches on partial-credit calibration, track whether escalations identify genuine ambiguity, and collect structured usability feedback from reviewers. The audit log and recorded human overrides provide a natural source for building that benchmark over time, provided the data are governed appropriately.

VII. VISUALIZATIONS AND TABULAR REPRESENTATIONS

The report uses the four figures above to show the measured operational profile of the complete pipeline. Figure 1 separates input and output tokens, which explains why assignments with similar numbers of calls can still cost different amounts. Figure 2 shows orchestration overhead. Figures 3 and 4 turn the measured token totals into a clearly qualified illustrative cost scenario. These figures are more informative than a generic architecture diagram because they show behavior from actual end-to-end runs.

The product itself also exposes useful review representations: a grade trace with ACT, REASON, CRITIQUE, and REVISION steps; batch results that show which submissions need attention; an instructor review surface with score overrides; and a student-facing feedback and regrade interface. For a demonstration, screenshots from these live views would complement the report by showing how the trace and audit data are used by people, not just how they are stored.

VIII. LINKS TO THE CODEBASE AND CONCLUSION

The complete source code is available at github.com/1AlgoRythm/AI_grading_agent. The repository includes the Streamlit application, Docker and Compose configuration, requirements, shared contracts, textbook corpus, lane-oriented modules, fixtures, CI workflow, and pytest coverage. The README documents local execution, deployment, configuration, BYOK usage, and the limits of the current evaluation approach.

The AI Grading Agent demonstrates a practical approach to applying GenAI to educational assessment. Its value lies less in asking a model to assign a score than in the surrounding reliability design: context is curated and budgeted, objective claims are verified when possible, the initial decision is criticized, disagreement is bounded and escalated, feedback is grounded, and instructor actions are audited. The next research step is a human-labeled benchmark that can measure accuracy and calibration directly. Until then, the system is best understood as an accountable grading assistant whose strongest contribution is making automated judgment inspectable.